

4.字符串

1 自动机与KMP

请你查询自动机相关资料，并论述KMP算法与自动机之间的区别和联系。你可以结合如下实例进行回答：已知字符集中只有a,b,c三种字符。现有字符串模式P=ababaca，你可以构造其KMP算法的next数组和DFA(确定有限自动机)，并说明KMP算法和自动机运行之间的联系。

(答案合理即可)

其next数组为：[-1, 0, 0, 1, 2, 3, 0]。

DFA用于识别以模式P结尾的字符串。状态表示当前匹配的前缀长度，初始状态为0，接受状态为6。转移函数 $\delta(i, c)$ 定义为：如果字符c匹配P[i]，则转移到状态i+1；否则，转移到状态 $\delta(\text{next}[i], c)$ （基于next数组回退）。

其中转换表是：

- 状态0:
 - ◦ $a \rightarrow 1$ (匹配P[0])
 - ◦ $b \rightarrow 0$ (不匹配，回退到状态0)
 - ◦ $c \rightarrow 0$
- 状态1:
 - ◦ $a \rightarrow \delta(\text{next}[1], a) = \delta(0, a) = 1$ (不匹配P[1]='b'，但回退后可能匹配前缀)
 - ◦ $b \rightarrow 2$ (匹配P[1])
 - ◦ $c \rightarrow \delta(0, c) = 0$
- 状态2:
 - ◦ $a \rightarrow 3$ (匹配P[2])
 - ◦ $b \rightarrow \delta(0, b) = 0$
 - ◦ $c \rightarrow \delta(0, c) = 0$
- 状态3:
 - ◦ $a \rightarrow \delta(\text{next}[3], a) = \delta(1, a) = 1$ (不匹配P[3]='b'，但回退到状态1后处理)
 - ◦ $b \rightarrow 4$ (匹配P[3])
 - ◦ $c \rightarrow \delta(1, c) = 0$
- 状态4:
 - ◦ $a \rightarrow 5$ (匹配P[4])
 - ◦ $b \rightarrow \delta(\text{next}[4], b) = \delta(2, b) = 0$
 - ◦ $c \rightarrow \delta(2, c) = 0$

- 状态5:
 - $a \rightarrow \delta(\text{next}[5], a) = \delta(3, a) = 1$ (不匹配 $P[5]='c'$ ，但回退到状态3后处理)
 - $b \rightarrow \delta(3, b) = 4$
 - $c \rightarrow 6$ (匹配 $P[5]$)
- 状态6 (接受状态) :
 - $a \rightarrow \delta(\text{next}[6], a) = \delta(0, a) = 1$
 - $b \rightarrow \delta(0, b) = 0$
 - $c \rightarrow \delta(0, c) = 0$

KMP与DFA

KMP就是一种DFA，在KMP算法中，利用NEXT数组模拟了DFA的行为；处理数组的时候，DFA使用状态转移，而KMP使用next数组来进行跳转，本质上是相同的行为。

在空间上DFA需要存储整个转换表而kmp只需要一个数组，更加节省，而且next数组会进行优化，能够压缩DFA失败处理中的花费

2

编写算法伪码，对给定的长度为n的字符串str，返回其中不含有重复字符的最长子串及其下标位置。

如str = "abcabcbb"，那么输出为 "abc" 和 0 要求算法的时间复杂度为 $O(n)$

使用一个滑动窗口和哈希表来寻找

```

def find(s):
    if not s:
        return "", -1

    char_map = {}
    max_length = 0
    start_index = 0
    left = 0

    for right in range(len(s)):
        current_char = s[right]

        # 如果字符已存在且在当前窗口内
        if current_char in char_map and char_map[current_char] >= left:
            left = char_map[current_char] + 1

        char_map[current_char] = right

        # 更新最长子串信息
        current_length = right - left + 1
        if current_length > max_length:
            max_length = current_length
            start_index = left

    return s[start_index:start_index + max_length], start_index

# 测试
str = "abcabcbb"
result, index = find(str)
print(f"最长不重复子串: '{result}', 起始下标: {index}")
# 输出: 最长不重复子串: 'abc', 起始下标: 0

```

因为每个字符最多被访问两次（也就是左右指针各一次），且哈希表操作是 $O(1)$ 的，因此时间复杂度是 $O(n)$ ，空间复杂度是 $O(\min(m,n))$ ，期中 m 是字符集大小，最坏的情况下需要存储所有不同的字符串

3

给定两个字符串 S 和 R ，长度分别为 n 和 m （均以字符数组形式存储，可就地修改）。在一次从左到右扫描中，原地（ $O(1)$ 额外空间）删除 S 中所有字符 'b'（删除要级联，即删除某个 'b'，并删除所有出现的子串 "ac" 可能使得原本不相邻的 a 与 c 变为相邻，这些新形成的 "ac" 也必

须被删除——示例见下)。要求单次遍历 (只能从左向右扫一次), 最终把保留的字符移到 S 的左端, 并返回新长度。

思路说明

可以利用滑动窗口和栈的思想, 同时用尽可能小的空间来模拟栈的行为。具体思路如下:

- 使用两个指针: 读指针 j 遍历原字符串 S , 写指针 i 指向当前修改后字符串的末尾。
 - 对于每个字符 $S[j]$:
 - 如果是 'b', 则直接跳过, **不写入**。
 - 如果是 'a', 则写入当前位置 i , 并递增 i 。因为 'a' 可能被后续的 'c' 删除, 所以暂时保留。
 - 如果是 'c', 则检查前一个写入的字符是否为 'a' (即 $S[i-1]$ 是 'a')。如果是, 则说明形成 "ac" 子串, 需要删除 'a' (通过递减 i); 否则, 将 'c' 写入当前位置 i 并递增 i 。
- 这样, 写指针 i 之前的字符构成修改后的字符串, 最终返回 i 作为新长度。

伪代码

函数 `deletePattern(S)`:

输入: 字符数组 S , 长度为 n

输出: 修改后 S 的新长度

```
i ← 0 // 写指针初始化
for j ← 0 to n-1 do:
    if S[j] == 'b':
        continue // 跳过 'b'
    else if S[j] == 'a':
        S[i] ← 'a' // 写入 'a'
        i ← i + 1
    else if S[j] == 'c':
        if i > 0 and S[i-1] == 'a':
            i ← i - 1 // 删除前一个 'a' (形成 "ac" 删除)
        else:
            S[i] ← 'c' // 写入 'c'
            i ← i + 1
return i // 返回新长度
```

时间复杂度分析

时间复杂度是 $O(n)$, 其中 n 是字符串 S 的长度。因为算法仅对 S 进行一次遍历, 每个字符最多被处理一次 (也就是读指针 j 移动 n 次), 而写指针 i 的移动也是常数时间操作。所以复杂度是 $O(n)$ 的

4

给定模式字符串 P （长度 m ）和文本字符串 T （长度 n ），两者均只含普通字符（无通配符）。要求计算一个数组 $\text{first_pos}[1..m]$ ，其中：

- $\text{first_pos}[k]$ 是 T 中最小的起始下标 i （采用 0-based）使得 $T[i..i+k-1] == P[0..k-1]$ （即模式 P 的前缀长度为 k 的串在 T 中第一次出现的位置）；
- 如果模式 P 的前缀长度为 k 在 T 中根本未出现，则 $\text{first_pos}[k] = -1$

要求时间复杂度 $O(n + m)$ ，额外空间（除输入与输出） $O(m)$ （用于 lps / 辅助数组）。请给出思说明和伪代码。

思路

借用KMP算法的前缀函数（ lps 数组）来高效匹配，在匹配过程中，对于每个前缀长度 k ，记录它第一次完全匹配的位置。所以：

- 当我们在位置 i 匹配到长度为 j 的前缀时，意味着 $P[0..j-1]$ 在 $T[i-j+1..i]$ 出现
- 对于每个 j ，我们只记录第一次出现的位置。因为我们只关心第一次出现，一旦某个前缀的第一次位置被记录，就不再更新
- 使用KMP的 lps 数组确保匹配过程是 $O(n+m)$ 时间复杂度

伪代码

函数 `computeFirstPos(P, T)`:

输入:

`P[0..m-1]` - 模式字符串

`T[0..n-1]` - 文本字符串

输出:

`first_pos[1..m]` - 结果数组

// 初始化结果数组

for `k = 1` to `m`:

`first_pos[k] = -1`

if `m == 0`:

`return first_pos`

// 计算KMP的lps数组

`lps = computeLPS(P)`

`j = 0` // `P`的当前匹配位置

for `i = 0` to `n-1`:

// KMP匹配

while `j > 0` and `T[i] != P[j]`:

`j = lps[j-1]`

if `T[i] == P[j]`:

`j = j + 1`

// 只记录当前匹配长度的第一次出现

if `first_pos[j] == -1`:

`first_pos[j] = i - j + 1`

`return first_pos`

复杂度

- 计算lps数组需要 $O(m)$ 的时间，因为每个字符最多被比较两次（前进和回退）
- 而匹配过程想要 $O(n)$ 的事件，因为文本`T`的每个字符最多被处理一次。而且由于lps数组的使用，模式`P`的指针`j`的回退操作不会增加总时间复杂度
- 因此一共是 $O(n+m)$ 的时间复杂度